

## Part 2: Conflict Identification and Resolution

### 1. Original Grammar Conflicts

The original grammar contains **10 conflicts** distributed across two main issues:

#### A. Dangling Else (Shift/Reduce in State 102)

```
[31] conditional_stmt -> IF '(' condition ')' stmt .
[32] conditional_stmt -> IF '(' condition ')' stmt . ELSE stmt
```

When parsing nested IF statements, encountering an ELSE creates ambiguity: should it attach to the inner or outer IF? Bison implicitly resolves this by shifting (attaching to the innermost IF), but this behavior is not explicitly defined in the grammar.

#### B. Block Ending Ambiguity (Reduce/Reduce in State 107)

```
[26] stmt_block -> '{' stmt_list '}' .
[33] repetition_stmt -> REPEAT expr '{' stmt_list '}' .
```

Both standalone blocks and repeat loops end with the identical sequence { stmt\_list }. Upon seeing a closing brace, the parser loses the context of whether a REPEAT expr preceded it. Bison defaults to reducing the first rule (stmt\_block), which breaks the parsing of all repetition loops.

### 2. Grammar Modifications

To resolve these conflicts, the grammar is explicitly modified using precedence declarations and rule refactoring.

#### A. Fixing the Dangling Else

Add precedence rules to the declarations section, giving ELSE higher priority:

```
%nonassoc THEN
%nonassoc ELSE
```

Modify the conditional statement to use the THEN precedence:

```
conditional_stmt:
    IF '(' condition ')' stmt %prec THEN
  | IF '(' condition ')' stmt ELSE stmt
  ;
```

#### B. Fixing the Block Ambiguity

Rename stmt\_block to compound\_stmt and refactor repetition\_stmt to use this non-terminal instead of raw braces:

```
compound_stmt:
    '{' stmt_list '}'
  ;

repetition_stmt:
```

```
    REPEAT expr compound_stmt
  | REPEAT compound_stmt
  | REPEAT expr ';'
;
```

### 3. Resolution Mechanisms

#### Dangling Else Resolution:

By declaring `%nonassoc ELSE` after `%nonassoc THEN`, `ELSE` is assigned a higher precedence. When the parser is in State 102 and sees an `ELSE` token, it compares the precedence of the lookahead (`ELSE`) with the production (`THEN`). Since `ELSE` is higher, the parser deterministically chooses to **SHIFT**, explicitly binding the `ELSE` to the nearest `IF`.

#### Block Ambiguity Resolution:

By extracting `{ stmt_list }` into a distinct `compound_stmt` non-terminal, we eliminate the overlapping suffix. The parser now immediately reduces `{ stmt_list }` to `compound_stmt` in a deterministic state. Only after this reduction does it look at the preceding context to decide whether to reduce to a standalone `stmt` or a `repetition_stmt`. The simultaneous completion of multiple productions is eliminated.

### 4. Before and After Parsing Examples

#### Example 1: Dangling Else

**Input:** `if (cond1) if (cond2) go 5; else turn south;`

- **Before Fix:** Parses correctly *only* due to Bison's implicit, undocumented fallback to `SHIFT`.
- **After Fix:** Parses deterministically via explicit grammar rules.

Parse Tree (Both Before and After):

```
if (cond1)
|- if (cond2)
   |- then: go 5;
   |- else: turn south;
```

#### Example 2: Repeat Loop with Block

**Input:** `repeat 10 { go 5; turn north; }`

- **Before Fix (FAILED):** The parser attempts to reduce the block as a standalone `stmt_block`, leaving the `REPEAT 10` as a broken, incomplete statement. The semantic meaning is lost.
- **After Fix (SUCCESS):** The block correctly reduces to a `compound_stmt`, which then completes the `repetition_stmt`.

Parse Tree (After Fix):

```
repetition_stmt
|- REPEAT
|- expr: 10
|- compound_stmt
```

```
| - stmt_list  
    | - go 5;  
    | - turn north;
```